Rime Hackathon Roadmap

Voice Agent with Continuity + Interruption Handling

Direction chosen: Conversation continuity during tool work + Interruption and recovery, applied to a voice-native booking/support agent. This is chosen because it is fundamentally a distributed-systems / state-management problem, which plays directly to a backend-systems skillset (Redis, WebSockets, async state, race conditions) rather than prompt engineering or UI polish.

If you decide to pivot to a different track (latency, telephony, multilingual, benchmark), the architecture in Sections 2–4 barely changes — only Section 6 (the hard problem) and Section 8 (evidence) change.

---

1 Problem Definition & Acceptance Test

Write this first, before any code.

User: Someone calling/talking to a booking or support agent (e.g., "check or change my appointment").

Problem: Two failure modes plague real voice agents:

1. User interrupts mid-response → old TTS audio keeps playing, or the agent "forgets" the interruption and finishes its old sentence anyway.
2. User interrupts *while a backend tool call is in flight* (e.g., a slow DB lookup) → the tool result comes back late and gets spoken as if it were the answer to the *new* question, or updates state the user never confirmed.

Claim to prove: *"When the user interrupts during agent speech or during a pending tool call, queued Rime audio stops within [X]ms, the stale tool result is discarded and never spoken or applied to state, and the agent's next utterance reflects only the user's latest request."*

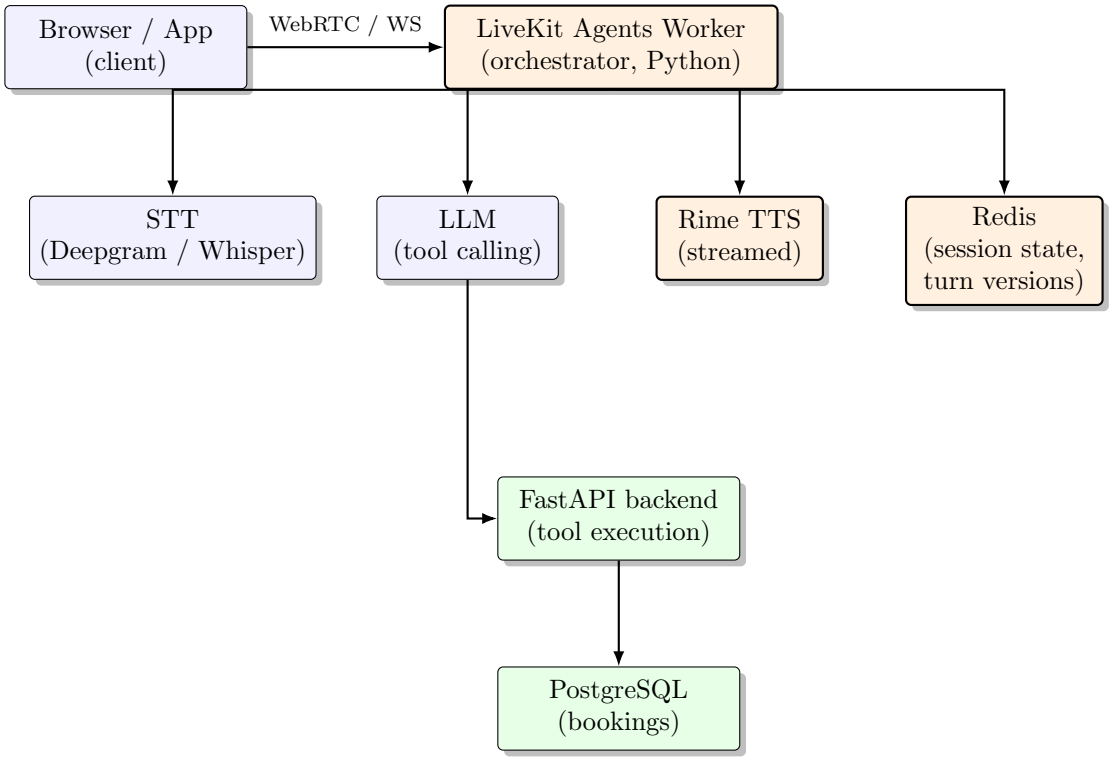
Acceptance test (define exact numbers before building anything):

- Inject a fixed artificial delay (e.g., 4000ms) into one tool call.
- At $t = 1500$ms into that call, simulate user speech that changes the request (e.g., "actually, change it to Friday" while it was still looking up Thursday).
- **Pass criteria:**
  - TTS playback of anything queued before the interruption stops within a defined threshold (e.g., < 300ms) of interruption detection.

- The original tool call's result, when it eventually arrives, is logged as "discarded — stale" and is never passed to TTS or written to the booking record.
- A fresh tool call reflecting the corrected request is issued and its result is what gets spoken.
- Final DB state matches only the corrected request.

Write this down as `RIME_EVIDENCE.md` §1 before writing a line of code. Everything below serves this test.

2 System Architecture



Component responsibilities:

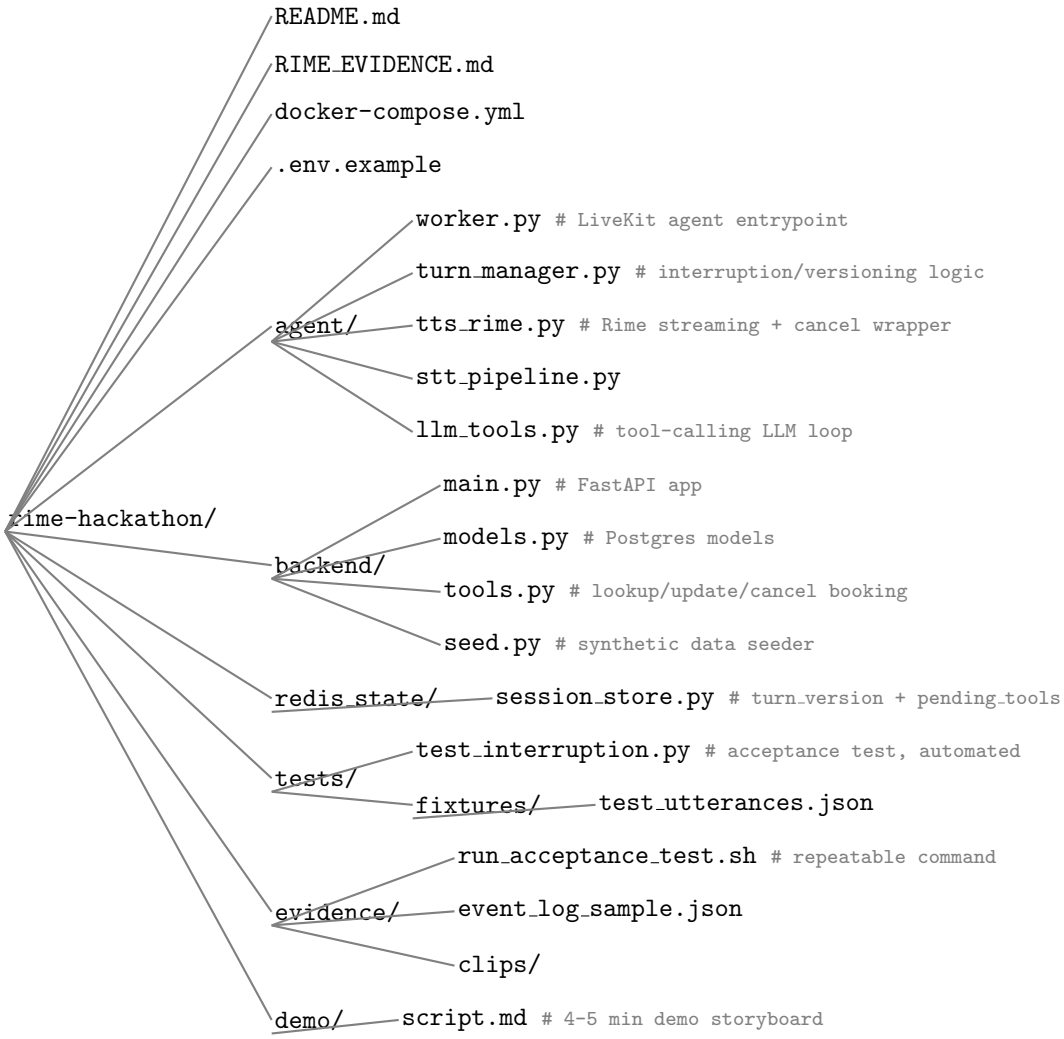
- **LiveKit Agents worker**: owns the session lifecycle, turn detection (VAD), and is the single place that knows "is the user currently speaking." This is where interruption detection hooks in.
- **STT**: converts user audio to text incrementally. *Partial* transcripts are needed, not just final ones, because interruption detection has to be fast — waiting for a "final" transcript is too slow.
- **LLM**: decides intent + calls tools (`lookup_booking`, `update_booking`, `cancel_booking`). Keep this simple — a single tool-calling loop. It is not the differentiator.
- **Rime TTS**: streamed audio output. Must be able to **cancel a stream mid-flight** — check the Rime/LiveKit plugin API for a cancel/stop method before committing to it.
- **Redis**: the actual differentiator. Holds per-session:
  - `session:{id}:turn_version` (integer, incremented on every detected interruption)

  - `session:{id}:pending_tools` (hash of tool_call_id → version tagged at dispatch)
  - `session:{id}:state` (last confirmed booking state, only mutated by non-stale results)
- **FastAPI backend**: executes tool calls against Postgres. Each tool call is invoked with the turn_version active at dispatch time, passed through and returned alongside the result so the orchestrator can check staleness on arrival.
- **PostgreSQL**: synthetic bookings table. Domain data, not training data — no ML dataset needed for this project.

3 Tech Stack Decisions

Layer	Choice	Why
Orchestration	LiveKit Agents (Python)	Explicitly recommended by the hackathon doc; handles turn detection, room/session lifecycle, and has an official Rime plugin — don't reinvent this
STT	Deepgram (streaming)	Fast partial transcripts, good LiveKit plugin support. Whisper local is a fallback if zero external STT dependency is preferred, at the cost of latency
TTS	Rime (required)	Use the official LiveKit-Rime integration; confirm cancel/interrupt support before building around it
LLM	Any tool-calling API model	Not the hard part — pick whichever API access is already available
Session state	Redis	Turn-versioning and pending-tool tracking are simple hash/string ops
Domain data	PostgreSQL	Relational fit for bookings
Backend API	FastAPI	Async-native, plays well with the async orchestration loop
Transport	WebRTC via LiveKit	Handles audio streaming, VAD, room management out of the box
Frontend (optional)	Minimal Next.js page w/ LiveKit client SDK	Just enough UI to join a room and show a live event log during the demo
Containerization	Docker Compose	Redis + Postgres + FastAPI + agent worker, for reproducibility
CI	GitHub Actions	Optional: run the interruption test harness (Section 7) on push

4 Repo Structure



5 Data & Fixtures (no ML dataset required)

5.1 Synthetic domain data

- `bookings` table: 30–50 synthetic rows — id, customer_name (fake), date, time, service_type, status.
- Generate with a simple script (Faker library or hand-written list) — never use real personal data.

5.2 Test utterance fixtures (`tests/fixtures/test_utterances.json`)

- Normal flow: "I'd like to check my booking for Thursday"
- Mid-tool-call interruption: same as above, but a second utterance fired 1.5s later: "actually make that Friday"
- Full cancel-and-restart: interrupt with an unrelated new request entirely

- Noise/garbage input: nonsense audio or silence, to test the STT/VAD boundary
- Rapid double interruption: interrupt twice in quick succession, to stress-test version bumping

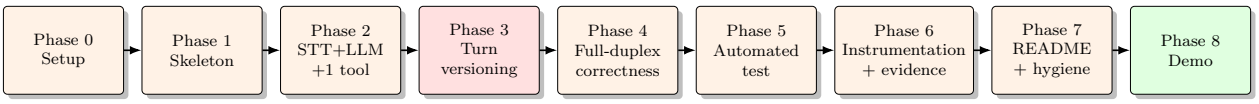
5.3 Injected delay fixture

A config flag/env var (`TOOL_CALL_DELAY_MS`) on `lookup_booking` so the race condition can be reproduced deterministically. This is not a hack — the doc explicitly wants controlled fault injection to prove the claim.

5.4 Event log schema (doubles as the evidence artifact)

```
{
  "session_id": "abc123",
  "events": [
    {"t_ms": 0, "type": "user_speech_start", "turn_version": 1},
    {"t_ms": 200, "type": "tool_call_dispatch", "tool": "lookup_booking", "turn_version":
         1, "tool_call_id": "tc1"},
    {"t_ms": 1500, "type": "user_speech_start_interrupt", "turn_version": 2},
    {"t_ms": 1520, "type": "tts_cancel", "reason": "interrupt", "latency_ms": 20},
    {"t_ms": 1600, "type": "tool_call_dispatch", "tool": "update_booking", "turn_version
        ": 2, "tool_call_id": "tc2"},
    {"t_ms": 4000, "type": "tool_call_result", "tool_call_id": "tc1", "
        turn_version_at_result": 1, "current_turn_version": 2, "action": "discarded_stale
        "},
    {"t_ms": 3800, "type": "tool_call_result", "tool_call_id": "tc2", "
        turn_version_at_result": 2, "current_turn_version": 2, "action": "applied"}
  ]
}
```

6 Build Phases (detailed)



Phase 3 (red) is the core differentiator; Phase 8 (green) is the deliverable.

Phase 0 — Setup

- Get Rime API key, confirm model/voice/language from the **live** catalog (not a cached list).
- Get Deepgram (or Whisper) access.
- `docker-compose.yml` with Redis + Postgres running locally.
- Confirm LiveKit's official Rime plugin supports mid-stream cancellation — **verify this first**, it is load-bearing for the entire hard-problem claim. If cancellation isn't natively supported, decide now whether to wrap the stream yourself (stop consuming/forwarding audio chunks client-side even if the server-side stream can't be killed).

Phase 1 — Skeleton: agent talks

- Minimal LiveKit Agents worker joins a room and speaks a canned sentence via Rime.
- Verify: exact model ID, speaker, language, endpoint, audio format, transport actually used — write these down now for the README.
- **Milestone:** join a room in a browser and hear Rime speak.

Phase 2 — STT + LLM + one real tool

- Wire streaming STT into the agent loop.
- Add one tool: `lookup_booking(booking_id)` → FastAPI → Postgres.
- No interruption handling yet — happy path only.
- **Milestone:** ask "what's the status of booking 12" out loud, hear the correct answer spoken back.

Phase 3 — Turn versioning in Redis (core differentiator)

- On agent start of speech/tool dispatch: read `turn_version`, tag everything downstream with it.
- On detected user speech during agent playback (VAD fires while TTS is streaming): increment `turn_version` in Redis, immediately stop/cancel TTS playback.
- On tool call dispatch: store `tool_call_id` → `turn_version` in `pending_tools` hash.
- On tool result arrival: compare its tagged version against current `turn_version`; if stale, log `discarded_stale` and drop it — never forward to TTS or a state-mutating write.
- **Milestone:** run the fixture in §5.2 manually and watch it behave correctly by ear + logs.

Phase 4 — Full-duplex correctness

- Confirm the application keeps accepting user audio *while* Rime is playing and *while* tools are running (not just after).
- Confirm background/cancelled work doesn't silently re-apply itself later (a cancelled tool call must not still write to Postgres).
- Add the rapid-double-interruption fixture and confirm versions don't get confused (off-by-one bugs are the most likely failure mode — write a unit test for version monotonicity).

Phase 5 — Automated acceptance test

- `tests/test_interruption.py`: spins up the fixture with injected delay, fires the interruption at $t = 1500$ms, asserts the event log matches expected outcomes (stale discarded, fresh applied, cancel latency under threshold).
- `evidence/run_acceptance_test.sh`: one command a judge can run to reproduce this.
- This is worth real points under "Evidence and reproducibility" — don't skip it for something only done live once.

Phase 6 — Instrumentation & evidence writing

- Make sure every event in §5.4's schema is actually logged during a real run, not synthesized after the fact.
- Write `RIME_EVIDENCE.md`: claim (from §1), acceptance test procedure, actual measured result (cancel latency, discard confirmation), limitations (e.g., "STT partial-transcript latency adds ∼150ms before interruption is even detected — this is a floor on reaction time, disclose it").

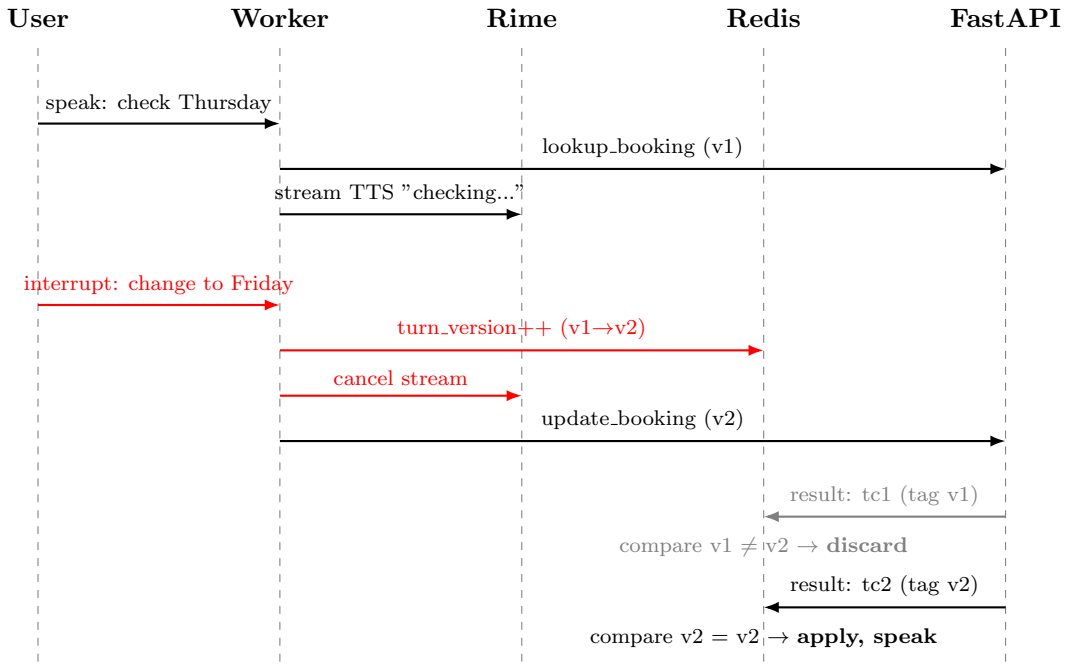
Phase 7 — README + config hygiene

- Exact Rime model ID/speaker/language/endpoint/audio format/transport.
- Setup instructions that actually work from a clean clone.
- `.env.example` with placeholders only.
- Explicitly state which speech provider is active and how a judge can verify it (e.g., a log line or a small status indicator in the demo UI).

Phase 8 — Demo script (4–5 min, storyboard first)

1. (30s) Who's the user, what's the problem.
2. (60s) Normal flow: ask about a booking, hear it answered correctly.
3. (90s) The stress case: trigger the delayed tool call, interrupt mid-flight with a changed request, show (a) audio stops immediately (b) the log shows the discard (c) the final DB state is correct.
4. (30s) Point at the active speech provider indicator.
5. (30s) One sentence on limitations, then stop.

Turn-versioning sequence (Phase 3 logic, visualized)



7 Testing Methodology Detail

For the automated test, structure assertions around three invariants — these are what "prove the claim" means concretely:

1. **Cancellation latency invariant**: `tts_cancel.t_ms - user_speech_start_interrupt.t_ms < threshold`
2. **No stale application invariant**: no `tool_call_result` with `turn_version_at_result != current_turn_version` ever has `action: applied`
3. **Final state correctness invariant**: DB state after the run matches only the last (post-interruption) request, never a merge of both

Run this test both "cold" (fresh containers) and "warm" (repeated calls) and report both — cold/warm must be distinguished, and unverified numbers get zero credit.

8 RIME_EVIDENCE.md Template

```
# Hard Voice Claim
[the claim from Section 1, verbatim]

# Acceptance Test
- Setup: [injected delay value, fixture used]
- Procedure: [exact steps, reference tests/test_interruption.py]
- Reproduce with: ./evidence/run_acceptance_test.sh

# Result
- Cancellation latency: measured Xms (cold), Yms (warm), N runs
```

```
- Stale discard rate: X/X correctly discarded
- Final-state correctness: X/X runs correct

# Limitations
- [STT partial-transcript latency floor]
- [any unsupported input class]
- [anything Rime-specific: e.g. no native mid-stream cancel,
  so client-side buffer drop was used instead -- disclose exactly what was done]
```

9 Submission Checklist

☐ Repo public/accessible, includes working demo link or recording (≤5 min)

☐ README with exact Rime model ID, speaker, language, endpoint, audio format, transport

☐ `RIME_EVIDENCE.md` complete with reproducible script

☐ `.env.example` — no real credentials anywhere (code, docs, screenshots, recording)

☐ Active speech provider observable in the running app

☐ Passed organizer's Rime config/secret preflight check

☐ Verified model/voice/language combo against the **live** catalog at submission time

☐ Synthetic data only in any booking/customer records

10 Risks / Likely Pitfalls

- **Rime stream cancellation might not exist server-side** — verify Phase 0 before building the whole turn-versioning system around an assumption.
- **VAD false positives** (background noise triggering "interruption") will corrupt the version counter — tune VAD sensitivity and test against the noise fixture explicitly.
- **Off-by-one version bugs** under rapid double interruption — write the monotonicity unit test early, not at the end.
- **Forgetting to disclose fallback behavior** — if STT/LLM/Rime has any fallback path, it must be visible, or the submission risks being flagged on eligibility grounds.

11 Copilot Prompts by Phase

Phase 1:

> ``Set up a minimal LiveKit Agents Python worker that joins a room and streams a canned sentence through Rime TTS using LiveKit's official Rime plugin.''

Phase 2:

> ``Add streaming STT (Deepgram) to a LiveKit agent worker, feeding partial and final transcripts to an LLM tool-calling loop with one tool, lookup_booking(booking_id), that calls a FastAPI endpoint backed by PostgreSQL.''

Phase 3:

> ‘‘Implement a Redis-backed turn-versioning scheme for a LiveKit voice agent: maintain turn_version per session, increment it when VAD detects user speech during agent TTS playback or during a pending tool call, tag every outbound TTS chunk and tool call dispatch with the version active at that moment, and on tool result arrival compare the tagged version to the current version --- discard (log only) if stale, apply if current.’’

Phase 4:

> ‘‘Write a pytest suite that spins up the FastAPI + Redis test environment, injects a 4000ms artificial delay into lookup_booking, simulates a user interruption at 1500ms with a changed request, and asserts: (1) TTS cancellation happens within 300ms of interruption detection, (2) the original tool result is logged as discarded and never applied, (3) the final booking state in Postgres reflects only the corrected request.’’

Phase 6:

> ‘‘Write a structured JSON event logger for a LiveKit voice agent session that records timestamped events (user_speech_start, tool_call_dispatch, tts_cancel, tool_call_result with turn_version tags) to a file per session, matching the schema in Section 5.4.’’

Phase 7 (Docker):

> ‘‘Write a docker-compose.yml with services for Redis, PostgreSQL, a FastAPI backend, and a LiveKit agent worker, with a seed script that populates 30 synthetic booking records on startup.’’